

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Proyecto

Sistema de Gestión de Inventarios basado en Microservicios para FerreAndina S.A.S

Entrega 1 ~ Investigación y fundamentación

Luis Felipe Giraldo Ortega
Steven Lopez Parra
Jhoan Sebastian Martinez Martinez

Universidad Nacional de Colombia
Departamento de Informática y Computación
Ingeniería de Software II
Manizales, Colombia

2026-2

1. Problema y alcance

Caso de estudio. FerreAndina S.A.S. es una distribuidora mayorista de ferretería, *ficticia*, con bodega principal en Manizales y bodegas en Pereira y Armenia. Maneja unas 3.800 referencias y 60 proveedores, y 15 personas trabajan con el inventario, entre administradores, jefes y operadores de bodega.

Problema. Cada bodega lleva su inventario en una hoja de cálculo que se consolida a mano una vez por semana. Por eso nadie sabe con certeza cuánto hay de cada producto en cada sede, las referencias de alta rotación se agotan sin aviso, los ajustes y traslados no dejan registro de quién los hizo, y las compras se deciden sin datos. Esto genera ventas perdidas por faltantes, compras innecesarias por desconocer el stock de otras sedes y falta de control interno.

Objetivo. Construir un sistema web de inventarios multi-bodega que registre cada movimiento de mercancía, alerte cuando un producto baje de su mínimo y sugiera cómo reabastecerlo (traslado desde otra sede o compra al proveedor), gestione las compras a proveedores y muestre indicadores del inventario.

Alcance. Incluye los requerimientos de la sección 5: productos y categorías, proveedores y órdenes de compra, movimientos de inventario por bodega, kardex, alertas, usuarios con roles, reportes y panel de indicadores. No incluye facturación electrónica, contabilidad, nómina ni punto de venta. El canal de venta en línea tampoco se construye; solo se deja una API de consulta de stock para integrarlo a futuro.

2. Sistemas similares

Se compararon tres sistemas: uno de código abierto (Odoo), uno SaaS internacional (Zoho Inventory) y uno SaaS colombiano (Alegra). Los datos salen de sus páginas oficiales [1–11].

Comparativo de sistemas de gestión de inventarios

Criterio	**Odoo Inventory**	**Zoho Inventory**	**Alegra**
Tipo y costo	Código abierto (LGPLv3) o SaaS; Standard 31,10 USD/usuario/mes	SaaS; desde 29 USD/mes (plan gratis limitado)	SaaS; plan Pyme 163.900 COP/mes
Multi-bodega	Sí	Sí, de 2 a 10 según plan	Sí, de 1 a 100 según plan
Órdenes de compra	Sí	Sí	Sí
Alertas de stock	Reglas de reabastecimiento	Puntos de reorden	Stock mínimo y máximo
Lotes y código de barras	Sí	Sí (plan Premium)	No publicado
Valoración	FIFO, costo promedio o estándar	No publicado	Valor del inventario
Arquitectura	Monolito modular (Python y PostgreSQL)	No publicada	No publicada
Fortalezas	Muy completo e integrado con todo el ERP	Fácil de empezar; fuerte en comercio en línea	En español y adaptado a la DIAN
Debilidades	Curva de aprendizaje; costo por usuario	Límites de órdenes y bodegas por plan	Inventario ligado a la facturación; bodegas limitadas por plan
Escalabilidad	Odoo worker como herramienta para manejar múltiples peticiones HTTP al tiempo	Escalabilidad horizontal automática y funciones serverless con pago por uso	Límites de uso según plan. Funcionamiento modular entre Contabilidad y sistema POS
Mantenibilidad	Monolito modular, un fallo en determinado módulo puede afectar al resto del sistema	Modelo SaaS; el mantenimiento y atención a incidencias corre por parte del equipo de Zoho	No publicado
Tolerancia a Fallos	Replicación de los datos y backups diarios en	Zoho promete una disponibilidad del servicio del 99,9%, con	No garantiza la disponibilidad de la aplicación.

Criterio	Odoo Inventory	Zoho Inventory	Alegra
	tres servidores diferentes.	respaldos de información en un servidor secundario.	

Conclusiones. Los tres coinciden en un núcleo: multi-bodega, órdenes de compra, alertas de reposición y trazabilidad de movimientos; ese es el mínimo del sistema propio. Los SaaS limitan bodegas y usuarios por plan, y Odoo es un monolito que escala replicando el servidor completo. El sistema propio se centra solo en inventario, sin límite de bodegas y con una API abierta para integrarse con otros canales. Además se consideran factores de calidad del software que denotan aspectos importantes que deben ser mejorados respecto a mantenibilidad y tolerancia a fallos, elementos críticos en arquitecturas monolíticas a diferencia de los microservicios.

Como elemento de valor respecto a las herramientas consultadas, ofreceremos un aplicativo modular priorizando el funcionamiento continuo ante fallos en dominios específicos. Esto garantiza un soporte organizado y eficaz, una disponibilidad completa materializada en servicios independientes y, valiéndonos de la arquitectura de microservicios, ofrecer calidad en el software presentado con la posibilidad de escalabilidad según la demanda.

El factor diferenciador más prominente es el **reabastecimiento inteligente entre sedes,** integrado y adaptado al caso multi-bodega**.** Los tres sistemas comparados avisan cuando un producto baja de su mínimo, pero la alerta solo dice que hay poco stock y deja la decisión al usuario. En el sistema propio, la alerta de stock bajo en una bodega también indica qué hacer:

- **Traslado interno:** si otra bodega (Manizales, Pereira o Armenia) tiene excedente del mismo producto, el sistema sugiere trasladarlo desde ahí.
- **Compra al proveedor:** si ninguna sede tiene excedente, sugiere generar una orden de compra al proveedor asociado al producto, con la cantidad calculada según el mínimo y la rotación reciente.

De esta forma, se resuelven varios problemas concretos: (**1**) Las referencias de alta rotación que se agotan sin aviso y las compras que se están decidiendo sin datos. (**2**) Evita compras innecesarias, porque primero se aprovecha el stock que ya está en otra sede. (**3**) Los SaaS comparados limitan bodegas y usuarios por plan. **Aquí es una función central, pensada para una distribuidora multi-bodega.**

Mientras los otros sistemas alertan cuando falta stock, el nuestro también sugiere la mejor acción según lo que hay en cada sede.

3. Monolito frente a microservicios

Comparación de arquitecturas para FerreAndina S.A.S.

Criterio	Monolito	Microservicios
Escalabilidad	Una sola base de datos y un solo stack para datos transaccionales y de reportes.	Cada servicio usa la base que le conviene: MySQL para saldos y órdenes, MongoDB para alertas y reportes.
Tolerancia a fallos	Un error en los reportes puede tumbar el registro de movimientos.	Si caen reportes o notificaciones, las bodegas siguen operando.
Despliegue	Cualquier cambio exige re-desplegar todo.	Cada servicio se despliega por separado.
Mantenibilidad	Los módulos terminan dependiendo unos de otros.	Cada servicio tiene su código y su base de datos.
Equipo	Todos trabajan sobre el mismo código, esto genera conflictos frecuentes.	Cada integrante trabaja en paralelo sobre sus servicios, con menos conflictos.
Complejidad	Baja: un proyecto y una base de datos.	Alta: red, gateway, contenedores y consistencia entre servicios.

Decisión. Se eligen microservicios. La consulta de stock es la operación más frecuente y conviene escalarla sola; el registro de movimientos no debe detenerse por fallas en reportes o notificaciones; y la empresa planea abrir más bodegas y un canal en línea. Se asume a cambio más complejidad de infraestructura, que se maneja con Docker y Kubernetes como exige el curso.

Otra de las razones para haber tomado esta decisión es la persistencia y tecnologías adecuadas por dominio:

En Monolito: Toda la aplicación comparte una única base de datos y un único stack, así que los datos transaccionales (saldos, movimientos, órdenes de compra) y los datos de consulta y análisis (reportes, alertas) se ven forzados al mismo modelo de almacenamiento.

En Microservicios: Cada servicio elige el almacenamiento que mejor encaja con su dominio: MySQL para Inventario, Compras, Catálogo y Usuarios, que exigen integridad transaccional, y MongoDB para Notificaciones y Reportes, que manejan documentos flexibles y consultas de lectura.

En FerreAndina los datos de inventario necesitan consistencia estricta, porque un saldo errado genera quiebres o sobrestock. Los reportes y las alertas, en cambio, necesitan flexibilidad y rapidez de lectura, no transacciones. Con microservicios cada dominio usa la tecnología que le conviene sin comprometer a los demás, y una consulta pesada de reportes no compite por recursos con el registro de movimientos en bodega. En un monolito habría que hacer concesiones para todo el sistema con una sola base de datos.

4. Microservicios preliminares

Cada servicio corresponde a un dominio del negocio (*bounded context*) y tiene su propia base de datos. El stack es preliminar y usa solo tecnologías vistas en clase.

Microservicios preliminares

Servicio	**Responsabilidad**	**RF**	**Stack**
API Gateway	Único punto de entrada: enruta, valida el token y aplica roles.	—	Laravel + MySQL
Usuarios	Usuarios, roles, inicio de sesión y contraseñas.	RF07, RF08	Express + MySQL
Catálogo	Productos, categorías, búsqueda y filtros.	RF01, RF02, RF12	Express + MySQL
Compras	Proveedores y órdenes de compra.	RF03, RF06	Express + MySQL
Inventario	Bodegas, movimientos, saldos, kardex y devoluciones.	RF04, RF05, RF10, RF13	Express + MySQL
Notificaciones	Alertas de stock bajo el mínimo.	RF09	Express + MongoDB
Reportes	Reportes e indicadores.	RF11, RF14	Express + MongoDB

Comunicación. Los clientes solo hablan con el gateway, y el gateway con los servicios, por REST. Entre servicios se usa REST cuando se necesita respuesta inmediata (por ejemplo, Compras valida un producto en Catálogo) y eventos asíncronos para lo demás: Inventario avisa a Notificaciones cuando un producto baja del mínimo y a Reportes cuando registra un movimiento. Para los eventos se propone RabbitMQ, pendiente de confirmar con el docente.

5. Requerimientos ~ Simplificados

En un documento aparte, están todos los requisitos tanto funcionales como no funcionales ampliados y con más detalle cada uno.

5.1 Funcionales

ID	Requerimiento
RF01	Crear, consultar, actualizar y dar de baja productos (código, nombre, descripción, categoría, unidad, precio y proveedor).
RF02	Gestionar categorías y subcategorías de productos.
RF03	Gestionar proveedores y asociarlos a uno o varios productos.
RF04	Registrar entradas, salidas, ajustes y traslados, y mantener el saldo de cada producto por bodega.
RF05	Administrar las bodegas y consultar el stock de un producto por bodega.
RF06	Crear órdenes de compra, seguir su estado y recibir mercancía contra ellas.
RF07	Gestionar usuarios con al menos dos roles (administrador y operador de bodega).
RF08	Iniciar y cerrar sesión, recuperar y cambiar la contraseña.
RF09	Alertar cuando el stock de un producto baje de su mínimo configurable.
RF10	Registrar quién, cuándo y qué movimiento se hizo (kardex).
RF11	Generar reportes de existencias, valorización y rotación.
RF12	Buscar y filtrar productos por nombre, código, categoría o proveedor.
RF13	Registrar devoluciones de clientes y a proveedores, reflejadas en el stock.
RF14	Mostrar un panel con indicadores: valor del inventario, quiebres de stock y rotación.
RF15	Exponer una API de consulta de stock para un futuro canal de venta en línea.
RF16	Sugerencia de reabastecimiento entre sedes.

5.2 No funcionales

ID	Requerimiento
RNF01	Arquitectura de microservicios independientes, comunicados por REST y eventos.
RNF02	Cada servicio tiene su propia base de datos; ninguno accede a la de otro.
RNF03	Todos los servicios y sus dependencias corren en contenedores Docker.
RNF04	Despliegue y gestión en Kubernetes.
RNF05	El servicio de inventario puede escalar de forma independiente.
RNF06	Las consultas responden en menos de 2 s con 50 usuarios concurrentes (15 internos más la API para otros canales).

ID	Requerimiento
RNF07	Autenticación con JWT y contraseñas cifradas.
RNF08	Cada servicio documenta su API con OpenAPI/Swagger.
RNF09	Toda petición pasa por el API Gateway propio; no se usa Kong ni gateways de terceros.
RNF10	Logs de cada servicio consultables desde un punto común.
RNF11	Cada integrante ejecuta al menos 15 pruebas de rendimiento.
RNF12	Repositorio en GitHub con actividad de todos los integrantes.
RNF13	Código con bajo acoplamiento y alta cohesión.
RNF14	Se despliega en cualquier entorno con Docker y Kubernetes sin cambiar el código.
RNF15	Solo tecnologías vistas en clase.
RNF16	Uso de IA documentado en la bitácora de cada entrega.
RNF17	Backup periódico de cada base de datos y procedimiento de recuperación ante fallos.
RNF18	Interfaz de registro de movimientos usable desde tablet o celular.

Anexo A. Bitácora de uso de IA - Entrega 1

Proyecto	Sistema de Gestión de Inventarios ~ FerreAndina S.A.S.
Equipo / Integrantes	Luis Felipe, Jhoan Sebastián, Steven Lopez
Entrega N.°	1
Periodo cubierto	15/09/2026 – 28/09/2026

Fecha	**Integrante**	**Herramienta**	**Tarea apoyada**	**Prompt (resumen)**	**Interv. humana**	**Resultado / aprendizaje**
19/09/2026	Felipe Giraldo	Claude	Plan de la entrega	Leer el enunciado y listar lo que pide la Entrega 1.	Medio	Lista de entregables. El equipo eligió una empresa ficticia.
21/09/2026	Sebastián Martínez	Claude + búsqueda web	Investigación de sistemas similares	Consultar funciones, precios y arquitectura de sistemas de inventario en sus páginas oficiales.	Medio	Datos para el comparativo (posteriormente rectificados). Lo que las páginas no dicen se dejó como “no publicado”.
23/09/2026	Felipe Giraldo	NotebookLM	Verificación del comparativo	Verificar cada celda contra las páginas oficiales cargadas en el cuaderno.	Alto	Encontró un precio de Odoo mal extraído (7,25 en vez de 31,10 USD); se corrigió. Aprendimos a no confiar en un solo resumen de IA.
24/09/2026	Sebastián Martínez	Claude + búsqueda web	Justificación, microservicios y ampliar requerimientos	Comparar monolito y microservicios para el caso, proponer servicios y ampliar los RF/RNF del enunciado con el formato de: Tipo, prioridad, origen, descripción, etc.	Alto	Al ser puntos tan importantes (sobre todo los requerimientos), se revisó uno a uno con detalle el cómo se amplió cada requisito, eliminando así información que está de más y adaptando cada uno a nuestro caso de estudio efectivamente.
26/09/2026	Steven López Parra	Claude	Simplificación del documento	Reducir el documento a lo que pide el enunciado para esta entrega.	Medio	Se decidió quitar el material de diseño (Saga, resiliencia, casos de industria) y dejarlo para la Entrega 2.

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt (resumen)	Interv. humana	Resultado / aprendizaje
27/09/2026	Steven López Parra	Claude	Investigación extra en las características de los sistemas consultados	Apoyo en la identificación de servicios de cada herramienta en materia de escalabilidad, mantenibilidad y tolerancia a fallos	Alto	Hubo una información mal referenciada en cuanto a Alegra y su tolerancia a fallos, mencionando que no había documentación al respecto. Salvo esta situación, la información fue veraz y contrastada manualmente.

Reflexión final de la fase

Usamos la IA para buscar y resumir información de sistemas existentes y para redactar el primer borrador. Las decisiones (empresa, alcance, qué dejar para la siguiente entrega) las tomamos nosotros. La principal limitación que encontramos fue un dato mal extraído por una herramienta, que detectamos al verificarlo con otra y contra la fuente original. También vimos que la IA tiende a producir **más de lo necesario**, así que recortamos el documento a lo que pide la entrega. Aprendimos que todo lo que genera la IA hay que verificarlo y entenderlo antes de presentarlo.

Referencias

[1] Alegra. *Software de inventario y facturación*. https://www.alegra.com/colombia/inventarios/.

[2] Alegra. *Planes y precios de Alegra en Colombia*. https://www.alegra.com/colombia/precios/.

[3] Alegra. *Cómo gestionar y configurar las soluciones de Alegra*. https://ayuda.alegra.com/int/mi-alegra-el-espacio-comun-de-todas-las-soluciones-en-alegra#p3.

[4] Alegra. *Alcance y nivel del servicio*. https://www.alegra.com/legal/terminos-y-condiciones-previos/2023-08-03/.

[5] Odoo S.A. *Odoo Inventory: Features*. https://www.odoo.com/app/inventory-features.

[6] Odoo S.A. *odoo/odoo*. https://github.com/odoo/odoo.

[7] Odoo S.A. *Pricing*. https://www.odoo.com/pricing.

[8] Odoo S.A. *The full stack platform*. https://www.odoo.sh/.

[9] Zoho. *Zoho Inventory features*. https://www.zoho.com/inventory/features/.

[10] Zoho. *Zoho Inventory pricing*. https://www.zoho.com/us/inventory/pricing/.

[11] Zoho. *Disaster recovery and business continuity*. https://www.zoho.com/security.html.

[12] NotebookLM. https://notebook.google.com

[13] Claude. https://claude.ai

Repositorio de Github del Proyecto

https://github.com/SebasMartinez21/Microservicios_FerreAndina